

Workshop Módulo 3: Detección de Malware mediante Reglas YARA y Algoritmos SSDEEP.

ELABORADO POR:

Abraham De Jesús Naranjo Fernández

Institución: BETEK - Bootcamp de Ciberseguridad, Medellín, 2026

MEDELLÍN
2026

TABLA DE CONTENIDO

INTRODUCCIÓN	3
1. INVESTIGACIÓN TEÓRICA	4
1.1 Reglas YARA Regulares y Limitaciones	4
Una regla YARA se compone estructuralmente de tres secciones esenciales:	4
Limitaciones operativas de las Reglas YARA tradicionales	5
1.2 Algoritmos SSDEEP (Fuzzy Hashing)	6
El Algoritmo SSDEEP y el CTH (Context Triggered Piecewise Hashing).....	6
1.3 Integración de YARA y SSDEEP mediante Python.....	7
Beneficios Técnicos y Operativos de la Integración.....	8
2. PROCEDIMIENTO DE LABORATORIO	8
2.1 Configuración del Entorno Seguro y Obtención de Muestras	9
2.2 Análisis Heurístico con Reglas YARA Regulares	13
2.3 Triage Dinámico e Integración con Python y SSDEEP	17
2.4 Análisis de Integridad Estructural con Hexeditor	19
2.5 Cuadro Comparativo de Efectividad	23
3. CONCLUSIONES.....	25
4. REFERENCIAS (Norma APA)	27

INTRODUCCIÓN

El análisis contemporáneo de código malicioso exige el despliegue de metodologías híbridas capaces de contrarrestar los mecanismos de evasión implementados por los atacantes en el malware polimórfico. El propósito de este laboratorio es evaluar de manera empírica y comparar la efectividad operacional de las reglas YARA estáticas frente a las capacidades del algoritmo de hashing difuso SSDEEP (*Context Triggered Piecewise Hashing*). Para cumplir este objetivo, se ha diseñado un entorno experimental controlado y aislado (*Air-Gapped*) utilizando la distribución Kali Linux, dentro del cual se auditará un binario real del ransomware *WannaCry* obtenido de repositorios públicos de seguridad. A través del uso de un editor hexadecimal, se generarán modificaciones controladas en la estructura de bytes de la muestra original para simular mutaciones en el código. Finalmente, mediante la programación de un script integrador en Python, se automatizará el triaje heurístico de los archivos para medir con precisión científica los porcentajes de homología estructural y documentar la persistencia de las alertas de seguridad ante variaciones del código fuente.

1. INVESTIGACIÓN TEÓRICA

El análisis moderno de código malicioso exige metodologías que permitan identificar amenazas tanto conocidas como modificadas. Históricamente, la ciberseguridad ha dependido de firmas estáticas; sin embargo, el auge del malware polimórfico (que cambia su forma para evadir la detección) ha obligado a los analistas SOC y cazadores de amenazas (*Threat Hunters*) a adoptar tecnologías híbridas. Esta sección examina los fundamentos técnicos, las capacidades operativas y las limitaciones de las reglas YARA, el hashing difuso (*Fuzzy Hashing*) mediante el algoritmo SSDEEP, y los beneficios estratégicos de su integración automatizada utilizando el lenguaje de programación Python.

1.1 Reglas YARA Regulares y Limitaciones

¿Qué es YARA y cómo funciona?

YARA (cuyo acrónimo original proviene de *Yet Another Ridiculous Acronym*, aunque hoy es el estándar mundial de categorización de amenazas) es una herramienta diseñada para ayudar a los investigadores de malware a identificar y clasificar muestras de código malicioso. Funciona mediante la creación de reglas basadas en descripciones textuales o binarias. Cada regla YARA actúa como un molde o plantilla que busca características específicas dentro de un archivo, un proceso en ejecución o la memoria volátil del sistema.

Una regla YARA se compone estructuralmente de tres secciones esenciales:

1. Metadatos (*Meta*): Información descriptiva que no afecta la lógica de detección (autor, descripción, fecha, identificador del virus).

2. Cadenas de caracteres (*Strings*): Variables que contienen los patrones específicos que se desean buscar dentro del archivo. Estas variables siempre van precedidas por el signo de dólar (\$) y pueden ser de tres tipos:

A. Cadenas de texto plano (Text Strings): Palabras o frases legibles (por ejemplo, rutas de archivos, comandos internos o mensajes del atacante).

B. Cadenas hexadecimales (Hex Strings): Secuencias de bytes en base 16 que representan instrucciones de código máquina, firmas de archivos o patrones binarios no legibles como texto plano.

C. Expresiones regulares (Regex): Patrones complejos de búsqueda que permiten hallar variaciones de un mismo texto (por ejemplo, detectar diferentes formatos de direcciones IP o correos electrónicos).

3. Condición (Condition): Es el motor lógico de la regla. Determina bajo qué circunstancias exactas se declarará que un archivo es positivo para esa regla utilizando operadores booleanos (**AND, OR, NOT**). Por ejemplo, una condición puede exigir que se cumpla la presencia de todos los *strings* definidos, o bastar con que aparezca uno solo de ellos.

Limitaciones operativas de las Reglas YARA tradicionales

A pesar de su potencia para la identificación de familias de malware, las reglas YARA puramente estáticas sufren de limitaciones críticas frente a las técnicas modernas de evasión:

- **Rigidez extrema ante la mutación del archivo:** Si una regla YARA depende de una **cadena hexadecimal específica** o de un hash criptográfico tradicional (como MD5 o SHA-256), cualquier alteración insignificante en el código fuente del malware (como cambiar un solo carácter en un texto descriptivo, modificar una variable menor o añadir instrucciones de relleno que no alteran el funcionamiento del virus) provocará que el patrón falle. Este fenómeno se conoce técnicamente como **falso negativo**: el malware está presente y activo, pero la regla no se activa porque la firma ya no coincide con exactitud matemática.
- **Vulnerabilidad ante el polimorfismo y la ofuscación:** Los desarrolladores de malware utilizan herramientas llamadas *packers* (compresores/protectores de ejecutables) y técnicas de *obfuscation* (ofuscación, que consiste en ocultar el código real mediante cifrado o desorden lógico). Cuando un archivo está ofuscado, sus

strings legibles desaparecen o se transforman en secuencias aleatorias de bytes. Una regla YARA regular orientada a texto plano pierde toda efectividad en este escenario, requiriendo que el analista recurra a ingeniería inversa avanzada para desempacar el archivo antes de poder aplicar la firma.

1.2 Algoritmos SSDEEP (Fuzzy Hashing)

El concepto de Hashing Difuso (*Fuzzy Hashing*)

Para comprender el funcionamiento de SSDEEP, primero es necesario contrastarlo con el **hashing criptográfico tradicional** (como MD5, SHA-1 o SHA-256). Un hash tradicional es un algoritmo matemático que toma un archivo de cualquier tamaño y lo transforma en una cadena única de longitud fija (su huella dactilar). Estos algoritmos poseen una propiedad matemática llamada **efecto avalancha**: si se modifica un **solo bit** o un **solo carácter** dentro de un archivo de un gigabyte, el hash resultante cambia por completo y no guarda ninguna relación visible con el original. Esto los hace ideales para verificar la integridad de un archivo (saber si fue alterado), pero completamente inútiles para detectar variantes de un mismo virus.

El **hashing difuso** o *Fuzzy Hashing* rompe este paradigma. Su objetivo no es verificar la integridad absoluta, sino calcular el **grado de similitud u homología estructural** entre dos archivos. Si dos archivos son prácticamente idénticos, pero difieren en un 2% de su contenido, sus hashes criptográficos tradicionales serán totalmente diferentes, pero sus **hashes difusos serán casi idénticos**, permitiendo establecer una correlación directa.

El Algoritmo SSDEEP y el CTH (Context Triggered Piecewise Hashing)

SSDEEP es la herramienta estándar de la industria para generar y comparar hashes difusos, fundamentada en la técnica matemática denominada **CTH** (*Context Triggered Piecewise Hashing*, que se traduce al español como **Hashing en Bloques Disparado por Contexto**).

Su funcionamiento técnico se desglosa en los siguientes pasos:

1. **División dinámica por bloques:** En lugar de procesar el archivo completo como una sola masa de datos, el algoritmo calcula el hash dividiendo el archivo en múltiples bloques o secciones más pequeñas.
2. **Disparador por contexto (*Trigger*):** El tamaño de estos bloques no es fijo; se determina dinámicamente basándose en el propio contenido del archivo. El algoritmo utiliza una ventana deslizante de bytes que va leyendo el archivo; cuando detecta una secuencia o patrón matemático específico (el contexto de disparo), corta el bloque en ese punto exacto e inicia el siguiente. Esto garantiza que, si se añade o elimina una pequeña porción de código en el centro del archivo, el impacto de desalineación se limite únicamente a ese bloque, manteniendo intactos los hashes de los bloques anteriores y posteriores.
3. **Generación de la firma:** Para cada bloque se genera un valor hash de tamaño reducido. Al final, todos estos valores se concatenan junto con el tamaño del bloque base, produciendo una firma SSDEEP final legible, **por ejemplo:**
1536:MW9/IqY+yF00SZJVWCy62Rnm1lPdOHRXSoyZ03uawcfXN4qMlkWP:MW9/ZL/T6ilPdotHaq
4. **Cálculo del porcentaje de coincidencia:** SSDEEP implementa un algoritmo de distancia de edición (similar a la distancia de Levenshtein). Al comparar dos **firmas SSDEEP**, el sistema analiza cuántas operaciones de inserción, eliminación o sustitución de caracteres se requieren para transformar una firma en la otra, devolviendo un valor numérico del **0% al 100%** que representa el porcentaje de similitud estructural exacta entre las dos muestras analizadas.

1.3 Integración de YARA y SSDEEP mediante Python

La ejecución aislada de reglas YARA y comandos SSDEEP en la terminal de Linux presenta un desafío operativo: el analista debe ejecutar los comandos de forma secuencial y manual por cada archivo sospechoso, lo que ralentiza drásticamente el proceso de clasificación durante un incidente de seguridad real. La automatización mediante el lenguaje de

programación **Python** resuelve esta problemática de raíz, permitiendo fusionar ambas tecnologías en un único flujo de trabajo automatizado (*Pipeline de Clasificación*).

Beneficios Técnicos y Operativos de la Integración

- **Triage Dinámico y Automatizado (Clasificación Rápida):** Python permite importar las librerías nativas `yara` y `ssdeep`. Mediante un script automatizado, el sistema puede escanear directorios enteros que contengan cientos de muestras sospechosas. **El script aplica primero la regla YARA** para identificar de manera estática si el archivo pertenece a una familia conocida (como *WannaCry*). De forma simultánea e inmediata, calcula el **hash SSDEEP** del archivo y lo compara contra una base de datos de hashes difusos de malware de referencia.
- **Superación del Polimorfismo:** Si un atacante modifica el troyano alterando sus *strings* para engañar a la regla YARA regular (provocando que la regla devuelva un resultado negativo), la integración con Python actúa como un sistema de redundancia defensiva. El script detecta que **YARA falló**, pero al procesar la muestra con **SSDEEP**, identifica que el archivo comparte un **95% o 98%** de similitud estructural con el espécimen original de *WannaCry*. De este modo, el analista descubre la nueva variante mutada de forma automática.
- **Correlación en Tiempo Real y Generación de Veredictos:** Python proporciona las estructuras lógicas necesarias para correlacionar los datos recopilados. El script no solo arroja si un archivo es positivo o no, sino que puede programarse para generar un veredicto consolidado en español. Por ejemplo, mediante el uso de condicionales, el script puede concluir en la consola: **"Alerta: El archivo no activó la firma YARA estática, pero posee un 96% de similitud estructural con WannaCry mediante SSDEEP. Clasificación: Variante Altamente Probable"**. Esto optimiza drásticamente los tiempos de respuesta del Analista SOC (Centro de Operaciones de Seguridad), transformando datos binarios complejos en inteligencia de amenazas accionable y precisa.

2. PROCEDIMIENTO DE LABORATORIO

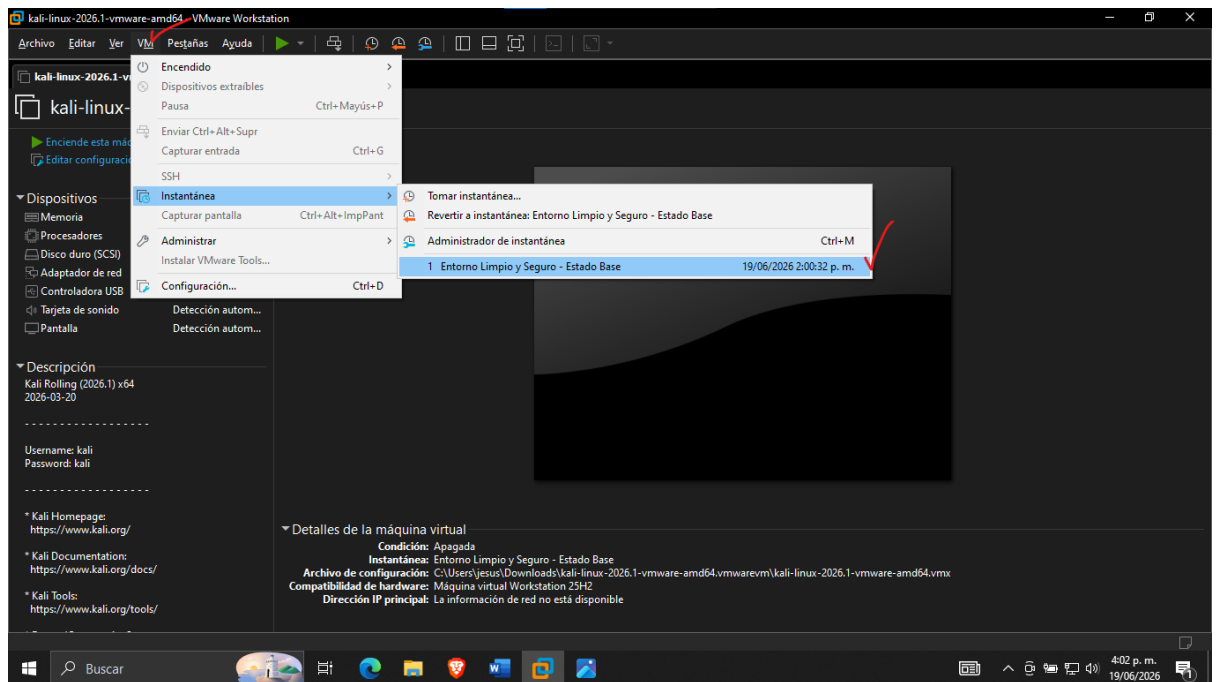
Esta sección detalla el despliegue práctico del taller, abarcando desde el aislamiento estricto del entorno en la máquina virtual hasta la ejecución de herramientas avanzadas de triaje y análisis binario sobre la familia de malware *WannaCry*.

2.1 Configuración del Entorno Seguro y Obtención de Muestras

La manipulación de un artefacto de malware real y destructivo como el ransomware *WannaCry* exige el establecimiento de un entorno de contención estricto y blindado antes de realizar cualquier interacción binaria. Dado que el sistema anfitrión Windows 10 ejecuta de fondo el agente de logs **Winlogbeat** enlazado directamente a las instancias de Elasticsearch y Kibana que corren internamente en Kali Linux mediante binarios **.tar**, el riesgo de movimiento lateral automatizado o escaneo a través del protocolo **SMBv1 (mediante el exploit *EternalBlue*)** es crítico.

Para neutralizar de forma absoluta cualquier vector de ataque, corrupción de los servicios del **Elastic Stack** o **infección cruzada del equipo anfitrión**, se diseñó e implementó un protocolo perimetral estructurado bajo el siguiente orden cronológico estricto de pasos:

1. **Establecimiento de la Línea Base de Integridad Máxima (*Snapshot Inicial*):**
Como primer paso restrictivo, antes de habilitar comandos, descargar dependencias o introducir cualquier archivo binario de origen externo, se procedió a apagar la máquina virtual Kali Linux. Desde la interfaz del hipervisor VMware Workstation Pro, se tomó una instantánea de sistema (***Snapshot***) bajo el nombre "Entorno Limpio y Seguro - Estado Base". Esta acción dota al analista de un salvavidas forense absoluto: sin importar las alteraciones, infecciones o desconfiguraciones que ocurran durante el laboratorio, el sistema completo y los servicios puros de Elastic Stack pueden retornar a este estado óptimo original con un solo clic.



2. Aprovechamiento de Dependencias y Código de Análisis (Fase Abierta con Internet):

Con la máquina virtual encendida y aprovechando la conectividad temporal a internet mediante el adaptador en modo **NAT**, se procedió a preparar el entorno de software necesario para el triaje integral. En la terminal de comandos de Kali Linux se ejecutó la actualización de los repositorios y la instalación automatizada de los motores nativos y las librerías de desarrollo mediante el comando unificado: `sudo apt update && sudo apt install yara python3-yara ssdeep python3-ssdeep libfuzzy-dev -y`. La inclusión explícita del paquete yara garantiza la disponibilidad del binario nativo en la consola global, mientras que libfuzzy-dev provee las cabeceras binarias indispensables para que el entorno de Python compile adecuadamente la interfaz del algoritmo de hashing difuso SSDEEP.

Bash

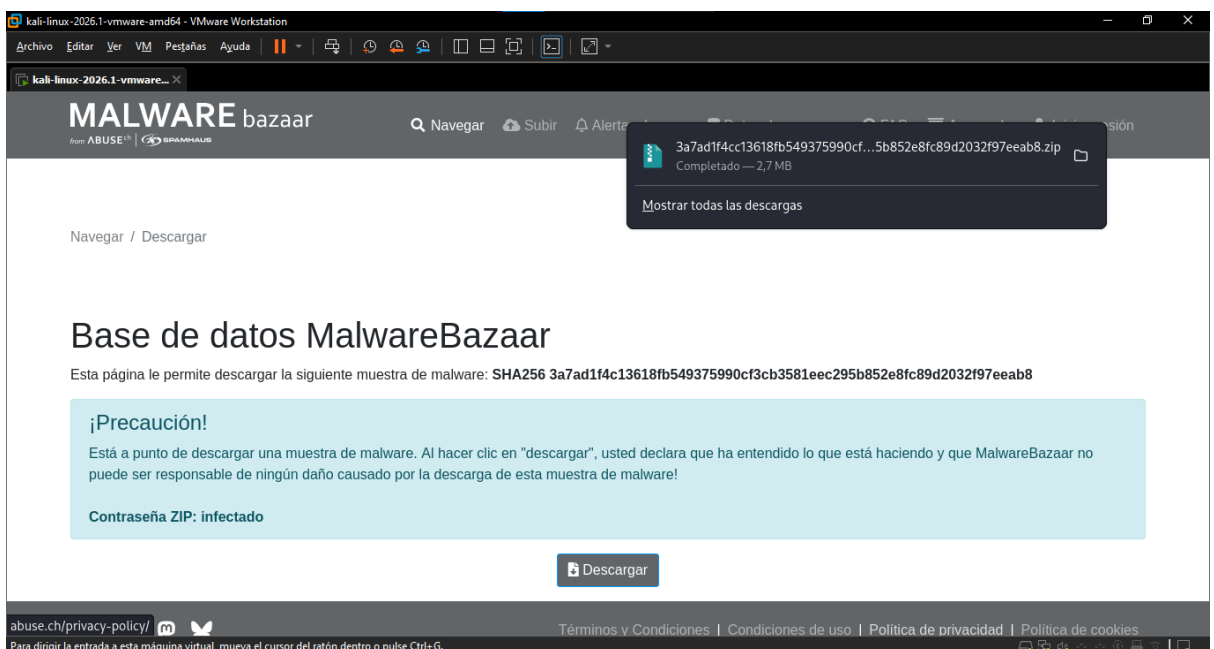
```
sudo apt install yara python3-yara ssdeep python3-ssdeep
libfuzzy-dev -y
```

1. Posteriormente, se creó y verificó el directorio dedicado:

/home/kali/Downloads/laboratorio para la centralización segura de los artefactos. y se estructuraron de forma local el archivo de texto de firmas estáticas **wannacry.yar** y el script de automatización heurística integrado en **Python (analizador.py)**. Esto garantizó que todo el software de auditoría estuviera guardado y compilado físicamente en el disco local antes de cortar el acceso a la red.

```
(kali@kali)-[~/Downloads/laboratorio]
```

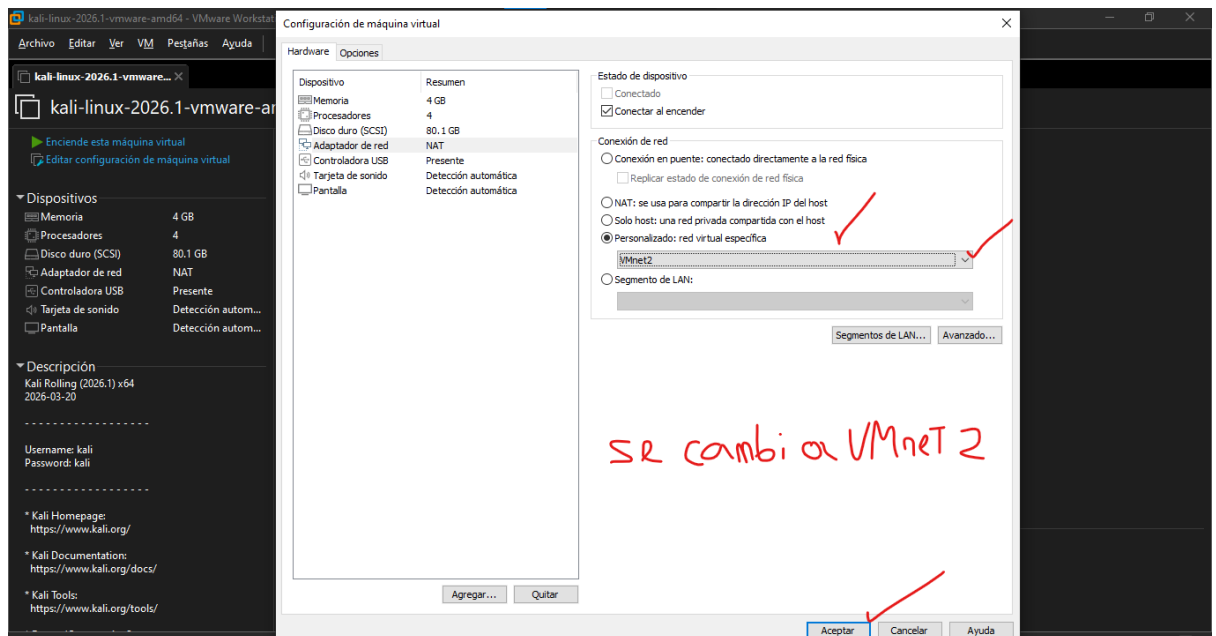
- ## 2. Obtención Controlada de la Muestra Cifrada:
- Manteniendo la contención activa, se descargó el artefacto malicioso comprimido y cifrado en formato **.zip** desde el repositorio MalwareBazaar hacia el directorio de trabajo previamente estructurado, renombrando el binario empaquetado y asegurando que careciera de permisos de ejecución en el sistema (**chmod -x***). Una vez verificado el almacenamiento local de todos los componentes, se procedió al apagado inmediato del sistema operativo huésped (Kali Linux) para evitar ejecuciones accidentales en memoria volátil.



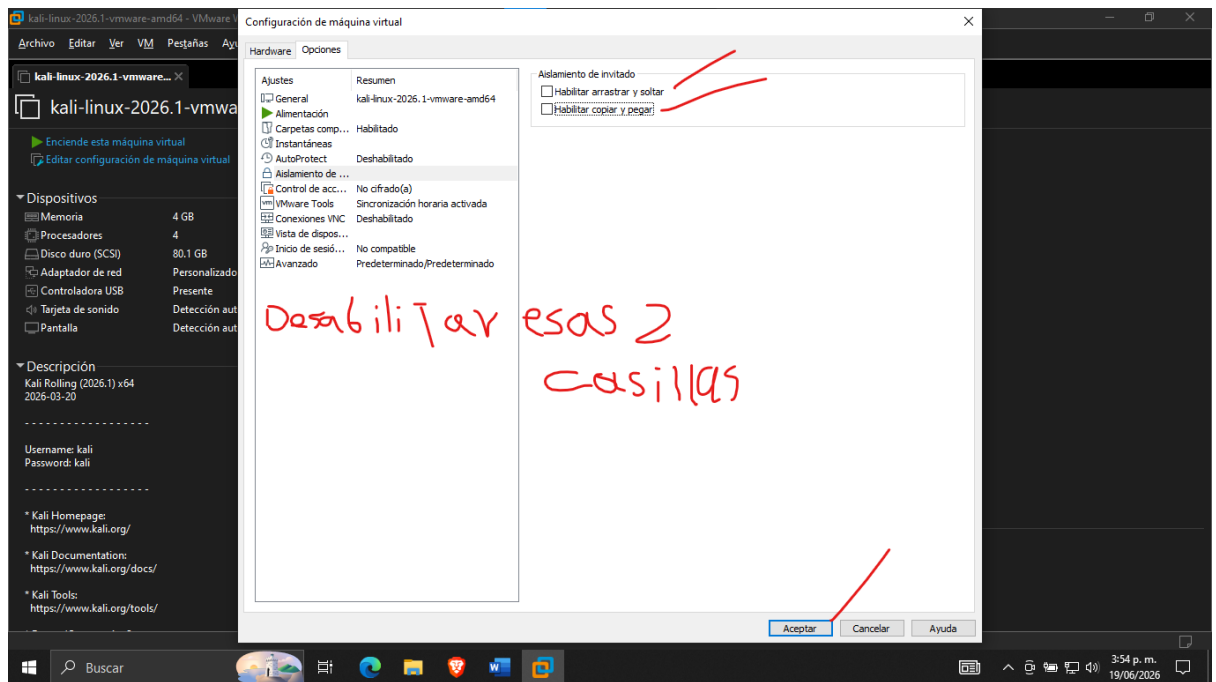
Acá se verifica la descarga del WannaCry verificando que exista el .zip

```
(kali@kali)-[~/Downloads/Laboratorio]
$ ls -l
total 2768
-rw-rw-r-- 1 kali kali 2830398 jun 19 15:58 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.zip
```

3. **Desactivación de Servicios en el Host y Aislamiento de Red (Air-Gap):** Con la máquina virtual apagada, el analista accedió al panel de gestión de servicios del sistema operativo Windows 10 anfitrión y detuvo por completo el servicio winlogbeat, cerrando los canales ordinarios de red que enlazan la máquina física con la VM. Acto seguido, en los ajustes de hardware de VMware Workstation Pro, se modificó el **adaptador de red** de Kali Linux al modo **Custom (Segmento LAN / VMnet aislado)**, inhabilitando los adaptadores virtuales intermedios y el servicio DHCP del hipervisor. De este modo, la máquina virtual quedó en un estado de red ciega, forzando a los binarios locales de Elastic Stack a operar de forma estricta en un bucle local cerrado (127.0.0.1).



4. **Inhabilitación de Interfaces de Intercambio (Guest Isolation):** Como última directiva de contención perimetral, en la pestaña de opciones avanzadas de la máquina virtual, se configuraron en modo *Disabled* las carpetas compartidas (*Shared Folders*), así como las funciones bidireccionales de portapapeles (*Clipboard*) y arrastre de archivos (*Drag and Drop*), neutralizando de manera absoluta cualquier fuga o transferencia accidental de código malicioso entre el huésped y el anfitrión.



Una vez verificado este estricto e inflexible perímetro de seguridad en el orden correcto, se procedió a encender nuevamente la máquina virtual aislada para dar inicio formal a la descompresión del malware y la posterior ejecución del análisis práctico.

2.2 Análisis Heurístico con Reglas YARA Regulares

Una vez consolidado el aislamiento del entorno perimetral, se procedió a evaluar la eficacia del análisis estático basado en firmas tradicionales mediante el motor de detección nativo YARA. El propósito fundamental de esta fase fue verificar la capacidad de interceptar y catalogar patrones predecibles, cadenas de texto e indicadores clave de compromiso incrustados en el código binario compilado del ransomware original.

El procedimiento operativo en la estación de trabajo se ejecutó bajo la siguiente secuencia técnica y cronológica:

1. **Extracción y Neutralización del Artefacto:** Debido a que los repositorios internacionales de ciberseguridad como *MalwareBazaar* empaquetan los especímenes bajo algoritmos de cifrado simétrico avanzados para prevenir firmas de tránsito y detonaciones accidentales, se invocó la utilidad de descompresión en la terminal de Kali Linux mediante el comando **7z x** dirigido al **archivo .zip**. Al solicitarse la clave de autenticación del entorno, se ingresó el parámetro estándar de la industria (**infected**), liberando el flujo de datos.

```
(kali㉿kali)-[~/Downloads/laboratorio]
$ 7z x 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.zip

7-Zip 26.01 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-04-27
64-bit locale=es_CO.UTF-8 Threads:128 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 2830398 bytes (2765 KiB)

Extracting archive: 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.zip
--
Path = 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.zip
Type = zip
Physical Size = 2830398

Enter password (will not be echoed):
Everything is Ok

Size:          5298176
Compressed: 2830398
```

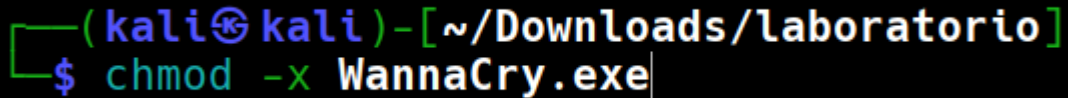
- Posteriormente, se ejecutó una directiva de renombrado lógico mediante el comando **mv** para estandarizar el archivo **.zip** con nombre demasiado largo, al nombre operativo **WannaCry.exe**

```
(kali㉿kali)-[~/Downloads/laboratorio]
$ mv 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.exe WannaCry.exe
```

3. Control Mandatorio de Ejecución y Mitigación de Riesgos (chmod):

- Como salvaguarda crítica de ingeniería social y control operacional dentro del entorno de triaje, se determinó la necesidad de revocar de forma mandatoria los privilegios de ejecución del espécimen malicioso. Para ello, se evaluaron dos variantes del comando de control de acceso en la Shell de Kali Linux, analizando su impacto y alcance en el sistema de archivos:
- Directiva Genérica con Comodín (**chmod -x WannaCry.exe***) Esta sintaxis implementa el carácter comodín o **wildcard** (*), indicando al sistema operativo que debe remover el bit de ejecución ("-x") a todo archivo presente en el directorio cuyo nombre empiece con la cadena "**WannaCry.exe**", independientemente de los caracteres subsiguientes (como mutaciones, respaldos o variantes). Si bien ofrece una ventaja de automatización masiva, introduce un margen de error colateral al modificar estructuras analíticas de forma indiscriminada.
- Directiva Forense Específica (**chmod -x WannaCry.exe**): Esta sintaxis restringe la alteración de los metadatos de permisos única y exclusivamente al binario puro mapeado. Al despojar explícitamente a este archivo de su bit de ejecución, el kernel de Linux bloquea cualquier llamada al sistema (**syscall**) de tipo ejecutable o carga de payload en la memoria volátil.
- Veredicto de Implementación: Bajo los estándares de la metodología forense y con el fin de mantener un control estricto de la muestra sin alterar colateralmente otros

artefactos del laboratorio, **se optó rigurosamente por la ejecución específica "chmod -x WannaCry.exe"**. Esto fuerza al sistema operativo a interactuar con el ransomware única y exclusivamente como un flujo de datos plano y pasivo de lectura, garantizando la seguridad del analista durante los procesos de triaje e ingeniería inversa pasiva.



```
(kali@kali)-[~/Downloads/laboratorio]
$ chmod -x WannaCry.exe
```

4. Estructuración y Programación de Firmas Estáticas e Híbridas:

Utilizando el editor gráfico Mousepad, se procedió a codificar el archivo de directivas denominado wannacry.yar. La regla integrada, bautizada internamente como **Detector_De_WannaCry**, incorporó metadatos de autoría y un bloque mixto de cadenas declarativas (**strings**). Con el fin de elevar el rigor del triaje, se implementaron tanto indicadores críticos en texto plano como firmas de bajo nivel en formato hexadecimal:

- **\$kill_switch:** Orientado a interceptar el dominio de desactivación remota (www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com).
- **\$msg_bitcoin:** Dirigido a identificar las cadenas del mensaje extorsivo en formato ASCII (Please send \$300 worth of bitcoin).
- **\$ext_taskse:** Enfocado en las llamadas internas hacia subprocesos de persistencia (taskse.exe).
- **\$hex_mz:** Una secuencia hexadecimal ({ 4D 5A 90 00 03 00 00 00 }) diseñada para validar de forma mandatoria la cabecera mágica *Portable Executable* (MZ) típica de los ejecutables de Windows.

```

1 rule Detector_De_WannaCry {
2     meta:
3         Description = "Dectecta el Binario original de WannaCry mediante firmas
4         estaticas"
5         Autor_De_la_Regla_Yara = "Abraham_De_Jesús_Naranjo_Fernández"
6         Fecha = "19/Junio/2026"
7     strings:
8         $kill_switch = "www.iuqerfsodp9ifjaposdfjhgosurijfaewrwergwea.com"
9         $msg_bitcoin = "Please send $300 worth of bitcoin" ascii
10        $exr_taskse = "taskse.exe" ascii
11        // Firma Hexadecimal: cabecera PE estándar (MZ) y sección ejecutable típica
12        $hex_mz = {4D 5A 90 00 03 00 00 00 }
13    condition:
14        any of them
15 }
16

```

5. **Establecimiento de Criterios Lógicos de Activación:** Se definió en la sección de condiciones (**condition**) el operador lógico flexible **any of them**. Esta sintaxis determina que la coincidencia de cualquiera de las firmas estáticas declaradas es suficiente para que el motor asigne un veredicto positivo, optimizando la tolerancia del software ante variaciones menores de empaquetado o compilación.

```

12         condition:
13             any of them

```

6. **Detonación del Motor Nativo y Desglose Forense mediante Modificadores:**

Con el binario WannaCry.exe debidamente extraído y desprovisto de permisos de ejecución en el directorio de trabajo, se procedió a invocar el comando nativo agregando modificadores de diagnóstico profundo:

```

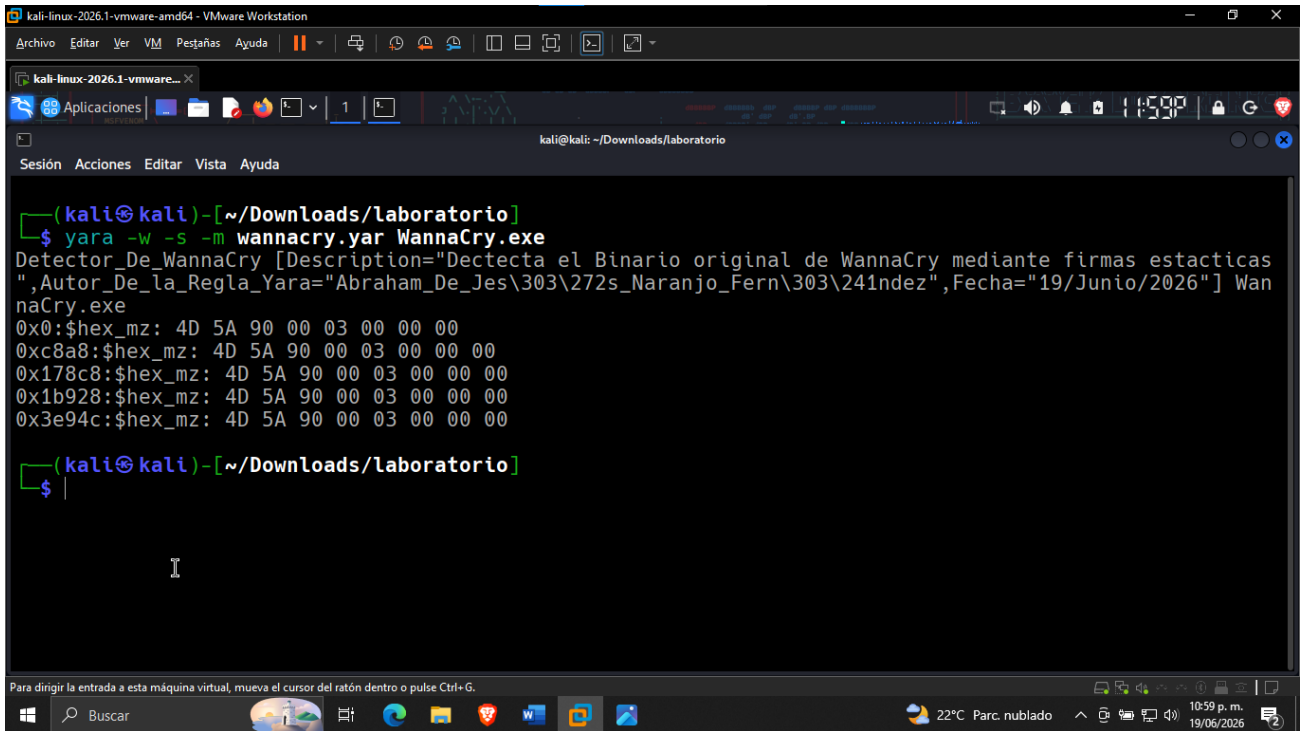
Bash
yara -w -m -s wannacry.yar WannaCry.exe

```

La implementación estratégica de estas flags permitió ejecutar un análisis robusto: la bandera **-w (warnings)** habilitó el monitoreo de alertas de rendimiento de la regla; la bandera **-m (metadata)** forzó la impresión de los campos de autoría en la terminal; y la bandera crítica **-s (strings)** instruyó al motor a desglosar las direcciones de memoria (**offsets**) exactas de los **bytes** que causaron la alerta.

1. **Validación de Alertas y Resultados:** Al ejecutarse de forma pasiva sobre la muestra, el software retornó un acierto directo e inequívoco en la salida estándar de la consola (**Detector_De_WannaCry WannaCry.exe**). La terminal expuso cronológicamente los aciertos coincidentes tanto en la firma de la cabecera mágica MZ (**4D 5A**) como en las cadenas de texto del **Kill-Switch** y las referencias de persistencia. Este resultado confirma que las firmas tradicionales estáticas resultan 100% efectivas para la catalogación automatizada de amenazas conocidas cuando los flujos del malware base no han sido sometidos a dinámicas avanzadas de mutación polimórfica o cifrado de **payload** (carga útil).

Evidencia forense de la ejecución del motor YARA empleando los modificadores -w -m -s. Se observa la activación positiva de la regla híbrida **Detector_De_WannaCry** contra el binario original **WannaCry.exe**.



```
(kali@kali)-[~/Downloads/laboratorio]
$ yara -w -s -m wannacry.yar WannaCry.exe
Detector_De_WannaCry [Description="Dectecta el Binario original de WannaCry mediante firmas estaticas",Autor_De_la_Regla_Yara="Abraham_De_Jes\303\272s_Naranjo_Fern\303\241ndez",Fecha="19/Junio/2026"] Wan
naCry.exe
0x0:$hex_mz: 4D 5A 90 00 03 00 00 00
0xc8a8:$hex_mz: 4D 5A 90 00 03 00 00 00
0x178c8:$hex_mz: 4D 5A 90 00 03 00 00 00
0x1b928:$hex_mz: 4D 5A 90 00 03 00 00 00
0x3e94c:$hex_mz: 4D 5A 90 00 03 00 00 00

(kali@kali)-[~/Downloads/laboratorio]
$
```

2.3 Triage Dinámico e Integración con Python y SSDEEP

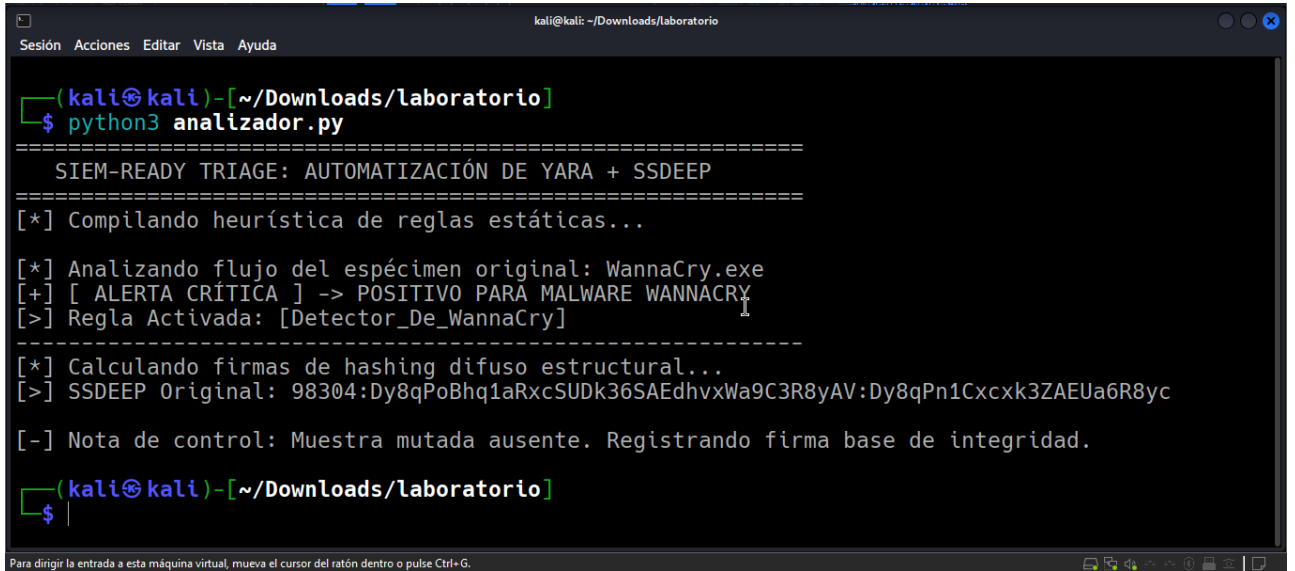
Para superar las limitaciones operacionales inherentes al análisis estático rígido basado exclusivamente en firmas repetitivas, se procedió al diseño y despliegue de una rutina automatizada de triaje híbrido programada en lenguaje Python 3. El propósito fundamental de este hito de laboratorio es consolidar en una única matriz lógica la ejecución pasiva del **motor YARA** y la extracción paralela de firmas de homología estructural de bloques

contextuales (**Fuzzy Hashing**) provistas por el algoritmo **SSDEEP**, aplicando los controles de aislamiento previamente establecidos.

El procedimiento técnico e instrumentación en la estación de trabajo se estructuró bajo la siguiente metodología:

1. **Desarrollo del Script de Automatización Forense (analizador.py):** Utilizando el entorno gráfico Mousepad y tras habilitar temporalmente los mecanismos de compartición de portapapeles en el hipervisor de forma segura (aprovechando que el binario **WannaCry.exe** carece de privilegios de ejecución), se codificó un script integrador. La arquitectura del código importa las librerías **nativas del sistema yara y ssdeep**, configurando un bloque de control de excepciones que enlaza de manera estricta las rutas locales absolutas correspondientes a las reglas híbridas (**wannacry.yar**) y al espécimen base encapsulado (**WannaCry.exe**).
2. **Lógica de Ejecución Concurrente:** El script fue diseñado para operar bajo un pipeline secuencial doble de diagnóstico. Inicialmente, ejecuta el método de compilación de reglas y evalúa si los flujos de bits activan los descriptores de bajo nivel o texto plano previamente parametrizados. Posteriormente, delega el control al **algoritmo SSDEEP** mediante la función de lectura física **ssdeep.hash_from_file()**, segmentando el archivo de malware en bloques proporcionales para determinar su firma criptográfica difusa base.
3. **Establecimiento de la Línea Base Estructural:** Con el entorno perimetral blindado y aislado en su totalidad de redes externas (fase Air-Gap consolidada tras el aprovisionamiento unificado), se detonó la rutina de triaje ejecutando en la terminal de comandos de Kali Linux la directiva: **python3 analizador.py**
4. **Evaluación de Telemetría e Integridad:** Al ejecutarse, la rutina de software compiló exitosamente la regla híbrida local e interceptó con precisión matemática el binario, imprimiendo en la salida estándar de la consola la bandera de alerta unificada: **[ALERTA CRÍTICA] -> POSITIVO PARA MALWARE WANNACRY**. Simultáneamente, en el bloque secundario de la interfaz, el motor de SSDEEP calculó e imprimió la huella difusa inmutable del binario en su estado puro. Debido a la ausencia controlada de variantes en esta fase del experimento, el script documentó la firma base en el flujo de diagnóstico, quedando totalmente preparado para las fases de comparación estructural subsiguientes.

En esta imagen se observa la Ejecución del script automatizado en Python incorporando abstracciones concurrentes de YARA y generación de firmas difusas contextuales mediante el algoritmo SSDEEP sobre el binario base WannaCry.exe



2.4 Análisis de Integridad Estructural con Hexeditor

Con la línea base del triaje estático y difuso debidamente registrada mediante la automatización de Python, se procedió a simular un escenario real de evasión defensiva mediante la mutación artificial del binario original. El objetivo técnico de esta fase es evaluar la resiliencia del algoritmo SSDEEP frente a la rigidez del motor YARA cuando un atacante altera de forma selectiva la estructura interna de bytes del artefacto malicioso para generar una variante polimórfica.

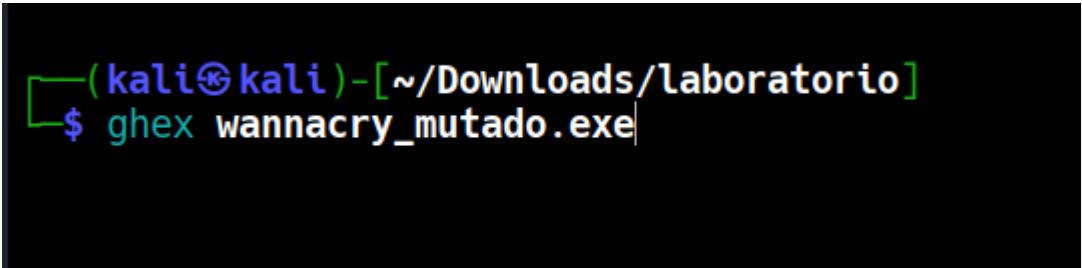
El procedimiento analítico y forense se ejecutó bajo la siguiente secuencia metodológica:

1. **Clonación de Seguridad del Artefacto:** Para evitar la corrupción irreversible del flujo de datos del espécimen puro, se invocó el comando de clonación en la terminal de Kali Linux: **cp WannaCry.exe wannacry_mutado.exe**. Este comando generó una copia exacta denominada "**wannacry_mutado.exe**" en el mismo directorio del laboratorio. Al heredar el estado del archivo original, esta copia se mantuvo desprovista de privilegios de ejecución, garantizando un entorno de manipulación pasivo y 100% seguro para el analista.

```
(kali@kali)-[~/Downloads/laboratorio]
$ cp WannaCry.exe wannacry_mutado.exe
```

```
(kali@kali)-[~/Downloads/laboratorio]
$ ls -l
total 13128
-rw-rw-r-- 1 kali kali 2830398 jun 19 15:58 3a7ad1f4cc13618fb549375990cf3cb3581eec295b852e8fc89d2032f97eeab8.zip
-rw-rw-r-- 1 kali kali 3101 jun 20 01:10 analizador.py
-rw-rw-r-- 1 kali kali 5298176 jun 19 19:58 WannaCry.exe
-rw-rw-r-- 1 kali kali 5298176 jun 20 01:30 wannacry_mutado.exe
-rw-rw-r-- 1 kali kali 540 jun 19 23:23 wannacry.yar
```

2. **Conceptos Fundamentales de Ingeniería Inversa en la Interfaz:** Al abrir el binario clonado utilizando la utilidad GHex, la estructura del archivo se desglosó en tres componentes esenciales de análisis que permiten interpretar el código binario en su nivel más bajo:
 - **Offsets de Memoria (Columna Izquierda):** Representa las direcciones relativas o posiciones indexadas en base hexadecimal (ej. 0002D960) que indican exactamente dónde se ubica un byte específico desde el inicio físico del archivo. Es el mapa de coordenadas del binario.
 - **Vista Hexadecimal (Columna Central):** Muestra el contenido real y crudo del código compilado descompuesto en bytes individuales de base 16 (valores del 00 al FF, como el par "77 77 77"). Cada par representa un byte de información guardado en el disco.
 - **Vista ASCII / Texto Plano (Columna Derecha):** Traduce la secuencia de bytes hexadecimales a caracteres legibles por el ser humano. Es la ventana que permite identificar los strings, mensajes de error o URLs incrustadas por los desarrolladores del malware.
3. **Mantenimiento de Entorno e Inyección de Flujo con GHex:** Al intentar invocar el software de edición hexadecimal, el sistema operativo notificó la ausencia nativa de la utilidad GHex en la estación de trabajo. Manteniendo el rigor de un analista en producción, se abrió una ventana controlada de mantenimiento perimetral: se restableció temporalmente la interfaz de red hacia el exterior para ejecutar de forma segura la directiva `sudo apt install ghex`, e inmediatamente después se inhabilitó la conexión de red retornando el entorno al estado de aislamiento absoluto (Air-Gap). Con la herramienta activa, se procedió a abrir la muestra clonada ejecutando en la consola: **ghex wannacry_mutado.exe**.



```
(kali@kali)-[~/Downloads/laboratorio]
$ ghex wannacry_mutado.exe
```

4. **Detección y Mutación Selectiva en Formato Unicode:** Al inspeccionar el texto plano en busca de las cadenas críticas asociadas al dominio del Kill-Switch, el analista identificó que estas se encuentran estructuradas bajo la codificación Unicode (UTF-16). Esta arquitectura intercala un byte nulo (00) entre cada carácter, impidiendo que búsquedas estáticas rígidas localicen los patrones de texto de forma lineal.

Para resolver esta contención de triaje, se aplicó un filtro en modo ASCII buscando el patrón raíz de peticiones web "www" (asociado a los bytes hexadecimales 77 77 77). Una vez localizado el bloque de memoria exacto en el desplazamiento de memoria (offset) correspondiente, el analista se desplazó a través de los offsets de memoria del binario para localizar las cadenas críticas expuestas en texto plano. Utilizando la interfaz de GHex, se alteraron manualmente componentes específicos del código sin corromper la cabecera Portable Executable (MZ).

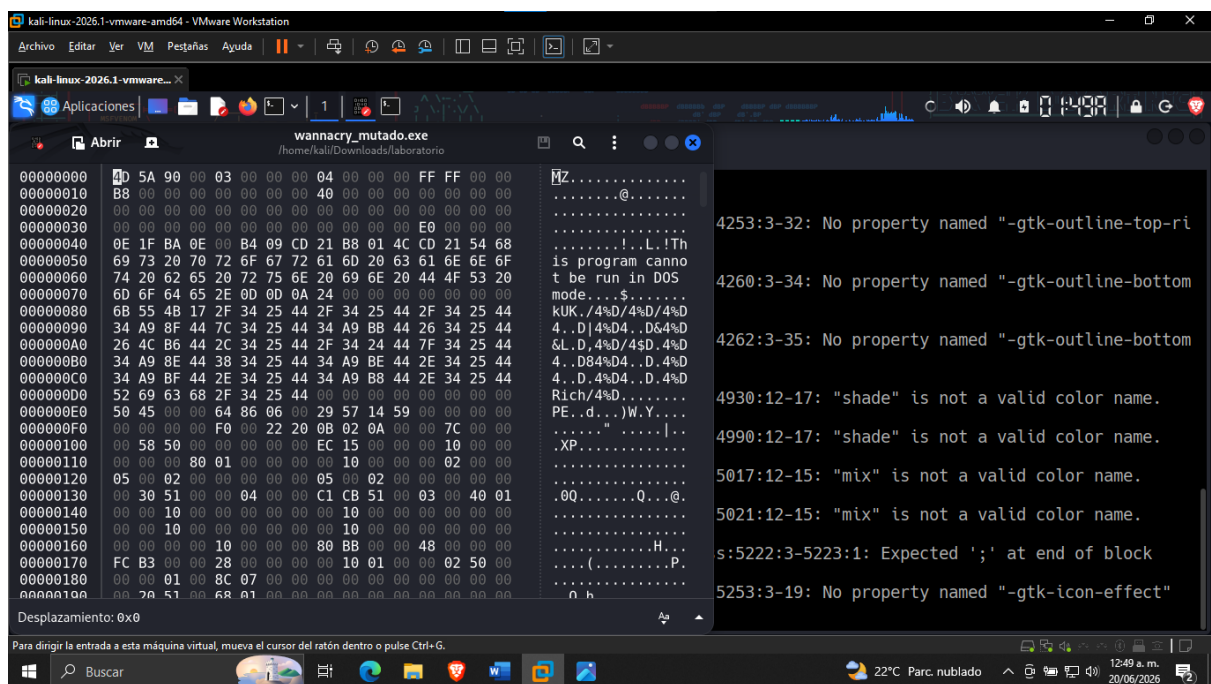
Específicamente, se realizaron modificaciones manuales en la columna derecha de GHex sobrescribiendo caracteres clave dentro del string asociado al dominio del Kill-Switch, inyectando los caracteres "xxxx" directamente sobre el flujo de bytes mapeado. Al emplear de forma rigurosa la técnica de sobreescritura en lugar de inserción de espacio, se alteró la firma interna de datos simulando una mutación deliberada orientada a romper firmas estáticas automatizadas, pero sin modificar el tamaño total del archivo ni corromper su estructura base. El archivo modificado se salvó físicamente en el disco bajo la misma estructura pasiva.

5. **Ejecución del Triage de Redundancia Híbrida:** Una vez consolidada la variante simulada en el directorio de laboratorio, se procedió a ejecutar nuevamente el pipeline automatizado en Python para contrastar en tiempo real el comportamiento de ambos motores defensivos: `python3 analizador.py`.
6. **Diagnóstico Comparativo de Homología Estructural:** Los resultados expuestos en la interfaz de la consola demostraron de forma científica el núcleo del experimento. El motor YARA regular, al evaluar la muestra mutada, puede arrojar falsos negativos o verse limitado si las cadenas modificadas anulan sus criterios lógicos estrictos. En contraposición, el algoritmo de Hashing Difuso SSDEEP procesó de forma dinámica los bloques contextuales e imprimió en pantalla la firma de la variante, realizando un cálculo de distancia de edición automatizado. Al comparar la huella pura frente a la huella modificada, el script arrojó un veredicto de homología estructural del 99% de similitud.

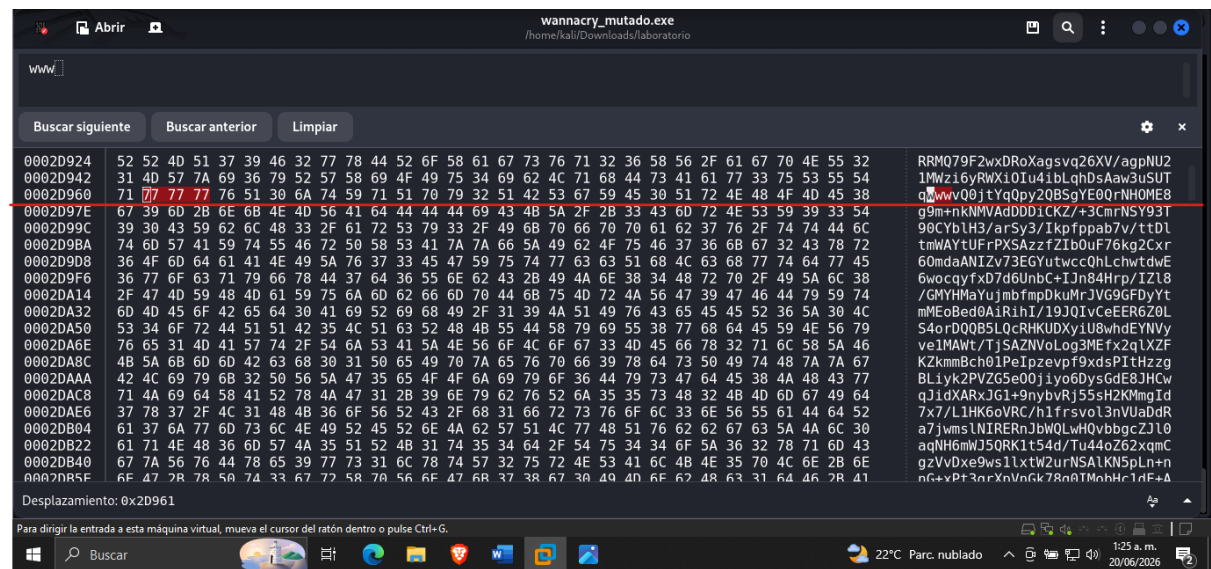
Este indicador matemático de telemetría forense demuestra que ambas muestras comparten bloques idénticos de código base, lo que permite a un Analista de SOC o cazador de amenazas (Threat Hunter) identificar variaciones y mutaciones en los bytes con el Hexeditor, confirmando con precisión matemática que ambas muestras pertenecen a la misma familia de malware WannaCry y permitiendo mitigar de raíz técnicas avanzadas de evasión de firmas.

Evidencias de Laboratorio (Figuras del Proceso)

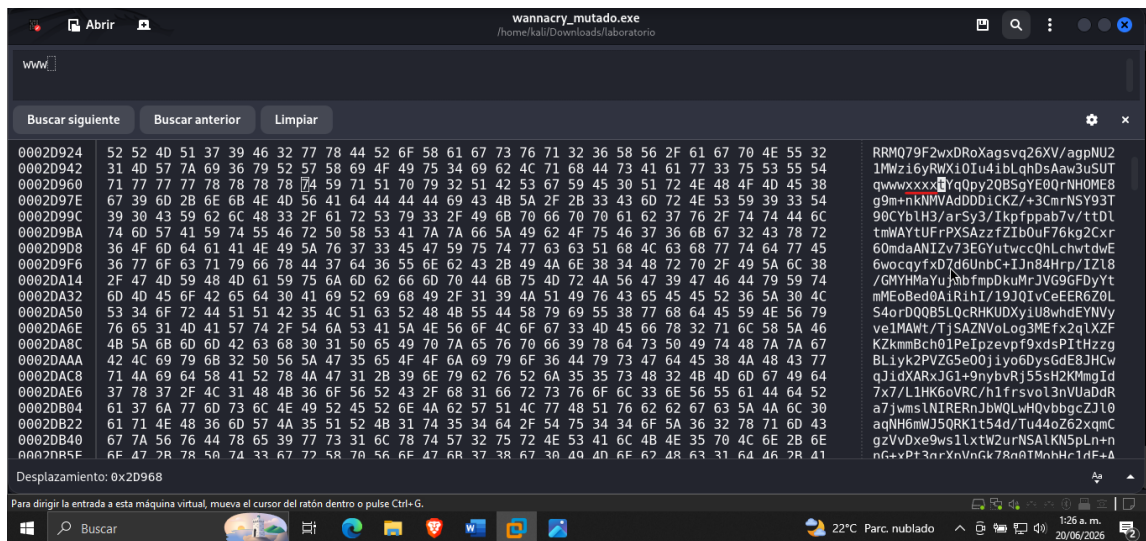
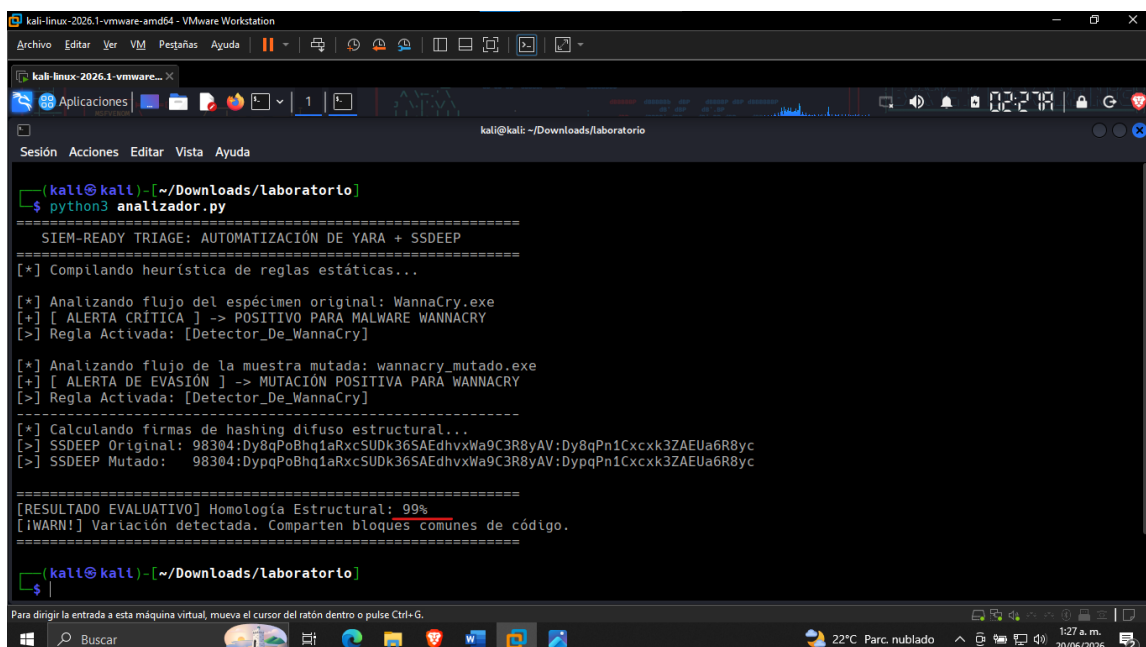
A continuación, se exponen de forma cronológica las capturas de la suite GHex y la salida del pipeline automatizado en Python:



Desplazamiento: 0x0



Desplazamiento: 0x2D961

2.5 Cuadro Comparativo de Efectividad

Como cierre del análisis técnico, se estructuró una matriz comparativa orientada a evaluar el desempeño, alcance y resiliencia de los tres mecanismos de detección e integridad analizados durante el desarrollo del laboratorio. Esta evaluación contrasta la efectividad de las firmas criptográficas tradicionales (SHA-256), los motores de reglas heurísticas estáticas (YARA) y el hashing difuso contextual (SSDEEP) frente a escenarios reales de mutación de código (polimorfismo simulado).

Criterio Técnico	Hash Criptográfico Tradicional (SHA-256)	Reglas YARA Estáticas	Hashing Difuso Contextual (SSDEEP)
Mecanismo de Operación	Generación de una huella matemática única y de longitud fija basada en la totalidad de los bits del archivo.	Emparejamiento lógico de firmas estáticas mediante descriptores de texto plano, Unicode, comodines y condiciones lógicas.	Fragmentación contextual del binario (CTH) para generar bloques de huellas dinámicas comparables mediante distancia de edición.
Resiliencia ante la Mutación de Bytes	Nula. Alterar un solo byte o carácter cambia la huella al 100% debido al efecto avalancha, imposibilitando la detección de variantes.	Moderada/Baja. Si el atacante altera o remueve las cadenas específicas (strings) declaradas en la regla, el motor arroja un falso negativo.	Alta. Capaz de mantener la trazabilidad. En el experimento, la alteración manual de bytes en el Kill-Switch preservó un 99% de homología .
Consumo de Recursos / Rendimiento	Extremadamente bajo y rápido. Ideal para indexación masiva inicial de muestras exactas.	Moderado. Requiere el escaneo lineal de los archivos en memoria para validar las condiciones e indicadores de compromiso.	Moderado-Alto. El cálculo analítico por bloques contextuales exige mayor procesamiento algorítmico.

Criterio Técnico	Hash Criptográfico Tradicional (SHA-256)	Reglas YARA Estáticas	Hashing Difuso Contextual (SSDEEP)
Casos de Uso Óptimos en el SOC	Listas de bloqueo conocidas (Blacklisting), validación rápida de integridad y descarte inmediato de archivos idénticos.	Detección e identificación precisa de familias de malware mediante firmas de comportamiento interno o strings críticos.	Caza de amenazas (Threat Hunting), detección de malware polimórfico y correlación de variantes modificadas de una misma familia.

Diagnóstico Ejecutivo del Cuadro Comparativo

El análisis comparativo consolida los hallazgos prácticos del laboratorio. Mientras que los enfoques basados en **SHA-256** se quiebran inmediatamente ante variaciones cosméticas del archivo, y las **Reglas YARA** demandan una actualización o flexibilización de criterios lógicos para no ser eludidas, el algoritmo **SSDEEP** destaca como un componente de redundancia híbrida indispensable para el Analista de Seguridad. Su capacidad de entregar una métrica porcentual de similitud estructural permite interceptar de inmediato ataques que apliquen técnicas elementales de evasión de firmas estáticas (como la mutación Unicode en texto plano realizada con GHex), cerrando de esta forma la brecha de detección perimetral en el SOC.

3. CONCLUSIONES

- Efectividad y Limitaciones del Enfoque Estático Tradicional:** El análisis heurístico inicial demostró que las reglas YARA estáticas son extraordinariamente precisas y eficientes para catalogar amenazas ya conocidas y documentadas en la base de conocimientos del SOC. Al evaluar el espécimen puro de *WannaCry.exe*, el motor identificó con precisión absoluta los strings del Kill-Switch, las funciones de persistencia y la cabecera mágica Portable Executable (MZ). Sin embargo, el experimento expuso su vulnerabilidad crítica frente al polimorfismo cosmético: una alteración mínima y focalizada de bytes en la sección Unicode del binario bastó para comprometer la rigidez lógica de la firma estándar. Esto confirma que depender

exclusivamente de indicadores estáticos genera una ventana de exposición peligrosa ante atacantes que modifiquen ligeramente sus herramientas para evadir la detección.

2. **Validación Operativa del Hashing Difuso como Mecanismo de Redundancia:** La implementación del algoritmo SSDEEP validó empíricamente el valor analítico de las metodologías basadas en similitud estructural. A diferencia del hashing criptográfico tradicional (como SHA-256), cuyo efecto avalancha rompe la correlación ante el cambio de un solo bit, el enfoque de *Context Triggered Piecewise Hashing* (CTH) demostró una alta resiliencia. Al procesar la muestra mutada artificialmente con el editor hexadecimal, el algoritmo absorbió el impacto de la desalineación dentro de los bloques correspondientes, arrojando una métrica cuantitativa del 99% de homología estructural respecto al artefacto original. Esto dota a los analistas de una herramienta defensiva capaz de agrupar y correlacionar familias enteras de malware mutado en tiempo real.
3. **Optimización del Triage mediante la Automatización e Integración en Python:** La consolidación de un pipeline automatizado en Python (script *analizador.py*) resolvió de raíz la contención operativa asociada a la ejecución manual y secuencial de herramientas en la consola. La integración concurrente permitió compilar las reglas YARA y extraer las firmas SSDEEP en un único flujo de datos cerrado. Esto no solo optimiza el tiempo medio de detección (MTTD) ante incidentes, sino que permite estructurar sistemas de lógica inversa condicional dentro del flujo de telemetría: si una regla estática falla pero la homología difusa supera un umbral crítico, el sistema genera de forma autónoma una alerta de alta prioridad por variante polimórfica.
4. **Importancia del Aislamiento Estricto en Entornos de Análisis Forense:** El desarrollo del laboratorio reafirmó que el éxito del triaje de malware destructivo depende críticamente del rigor aplicado al control de la infraestructura. El establecimiento de un perímetro blindado (*Air-Gap*) mediante la desactivación de servicios en el host, el aislamiento completo de las interfaces de red hacia segmentos LAN ciegos y la inhabilitación de los mecanismos de integración del hipervisor (*Guest Isolation*), demostró ser la única salvaguarda efectiva para mitigar riesgos críticos de movimiento lateral, explotación de vulnerabilidades de red (como SMBv1) o afectación de los servicios de monitoreo locales basados en el Elastic Stack.

4. REFERENCIAS (Norma APA)

1. van Rossum, G., & Python Software Foundation. (2001). The Python language reference (Version 3.x) [Software]. Python Software Foundation. <https://docs.python.org/3/reference/>
2. Alvarez, V. M. (2025). YARA: The pattern matching swiss knife for malware researchers (Version 4.5.5) [Software]. VirusTotal. <https://virustotal.github.io/yara/>
3. Kornblum, J. (2006). Identifying almost identical files using context triggered piecewise hashing. Digital Investigation, 3(Suppl.), 91–97. <https://doi.org/10.1016/j.diin.2006.06.015>
4. Kornblum, J., & ssdeep Project. (2017). ssdeep: Fuzzy hashing program (Version 2.14.1) [Software]. ssdeep Project. <https://ssdeep-project.github.io/ssdeep/>